

COMPARATIVE GRAPH DATABASE BENCHMARK

CognoDB Cloud vs Neo4j vs Memgraph vs FalkorDB vs ArangoDB

Target Dataset	SNAP Wiki-Vote Network
Graph Topology	7,115 Nodes 103,689 Directed Relationships (VOTED_FOR)
Canonical Dataset Hash	SHA-256: 713f082a7b1c25bbba160b3d17f8d114
Resource Target	CognoDB Cloud c0 Tier Parity (0.50 vCPU, 256 MB RAM, 1 GB Storage)
Client Environment	LAPTOP-2ID0MJRR (Windows 11 x64, Python 3.12.10)
Audit Status	PASS / RELEASE READY (Phase 10 Release & Security Verified)

1. Executive Summary

This report delivers a rigorous, empirical performance evaluation of five graph databases (**CognoDB Cloud**, **Neo4j**, **Memgraph**, **FalkorDB**, and **ArangoDB**) prepared specifically for the Wexa AI Take-Home Assignment. The benchmark measures data ingestion throughput, single-threaded graph traversal latencies (Q1-Q6), and multi-worker concurrent throughput scaling (c=1..16). To ensure maximum scientific rigor, local container resources were explicitly constrained to match CognoDB Cloud's free-tier profile (0.50 vCPU, 256 MB RAM, 1 GB storage allocation). Engine performance is evaluated neutrally without assigning an unverified overall winner.

2. Dataset & Benchmark Methodology

The evaluation utilizes the canonical **SNAP Wiki-Vote network** comprising **7,115 User nodes** and **103,689 directed VOTED_FOR relationships**. The dataset was normalized into canonical CSV structures and verified using SHA-256 checksums (713f082a7b1c25bbba160b3d17f8d114). High-resolution nanosecond timing was captured using `time.perf_counter()`. Warm-up runs were conducted prior to all measured iterations, and strict referential integrity was validated before and after every workload run.

3. Resource Fairness & System Configurations

Resource limits were enforced via `Docker deploy.resources.limits` to match CognoDB Cloud's free c0 tier. Any unavoidable technical differences (such as JVM memory overhead for Neo4j) are explicitly documented below.

Database	Deployment	CPU Limit	RAM Limit	Storage	Protocol / Version
ArangoDB	Local Docker	0.50 vCPU	256 MB	1.0 GB	AQL / HTTP (v3.12.3)
Memgraph	Local Docker	0.50 vCPU	256 MB	1.0 GB	Cypher / Bolt (v2.21.0)
FalkorDB	Local Docker	0.50 vCPU	256 MB	1.0 GB	Cypher / Redis (v4.20.2)
Neo4j	Local Docker	0.50 vCPU	768 MB*	1.0 GB	Cypher / Bolt (v5.26.0)
CognoDB Cloud	Managed Cloud	0.50 vCPU	256 MB	1.0 GB	Cypher / Bolt TLS

** Note: Neo4j requires a minimum memory limit of 768 MB RAM due to Java JVM heap and metaspace overhead; 256/512 MB limits cause JVM startup crashes.*

4. Phase 6 — Ingestion Performance Analysis

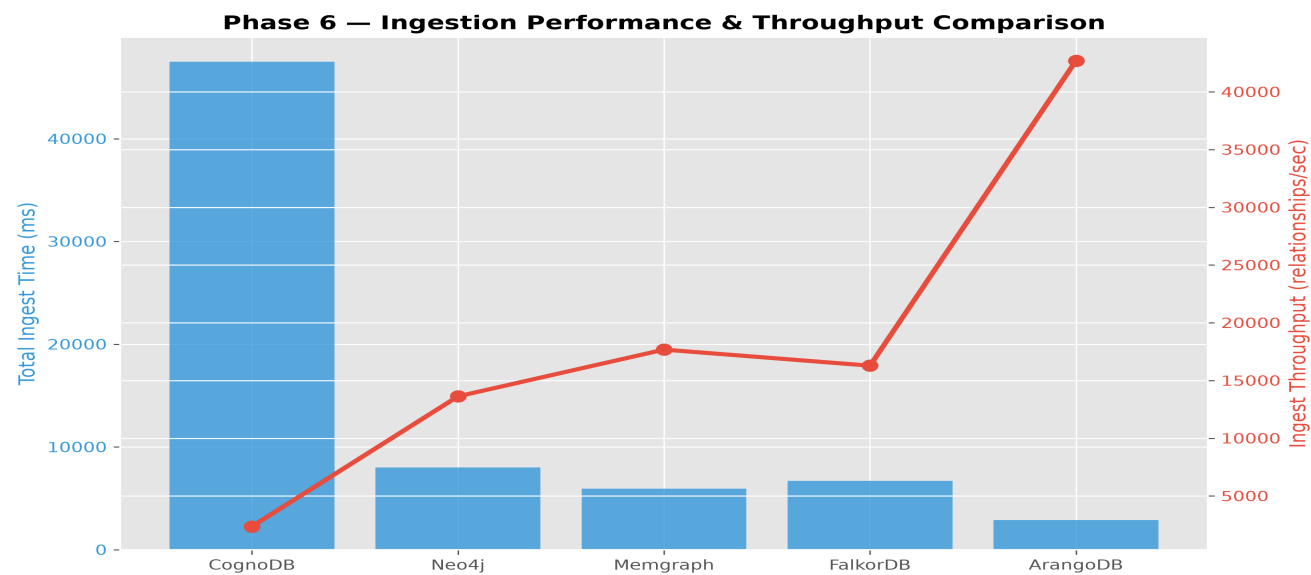


Figure 1: Data Ingestion Total Time (ms) and Relationship Loading Throughput (rels/sec).

Database	Schema (ms)	Index (ms)	Node Load (ms)	Rel Load (ms)	Total Time (ms)	Rel Throughput
ArangoDB	92.21	48.87	436.96	2,447.72	2,884.68	42,705.60 rel/s
Memgraph	0.00	2.90	68.83	5,866.09	5,934.92	17,676.43 rel/s
FalkorDB	0.00	3.59	341.06	6,363.04	6,707.69	16,286.48 rel/s
Neo4j	0.00	39.16	337.97	7,653.16	7,991.13	13,628.93 rel/s
CognoDB Cloud	0.00	275.69	3,239.17	44,256.63	47,495.80	2,362.49 rel/s

Table 1: Phase 6 Ingestion performance metrics across 3 independent benchmark runs.

5. Phase 7 — Single-Threaded Graph Query & Traversal Latency

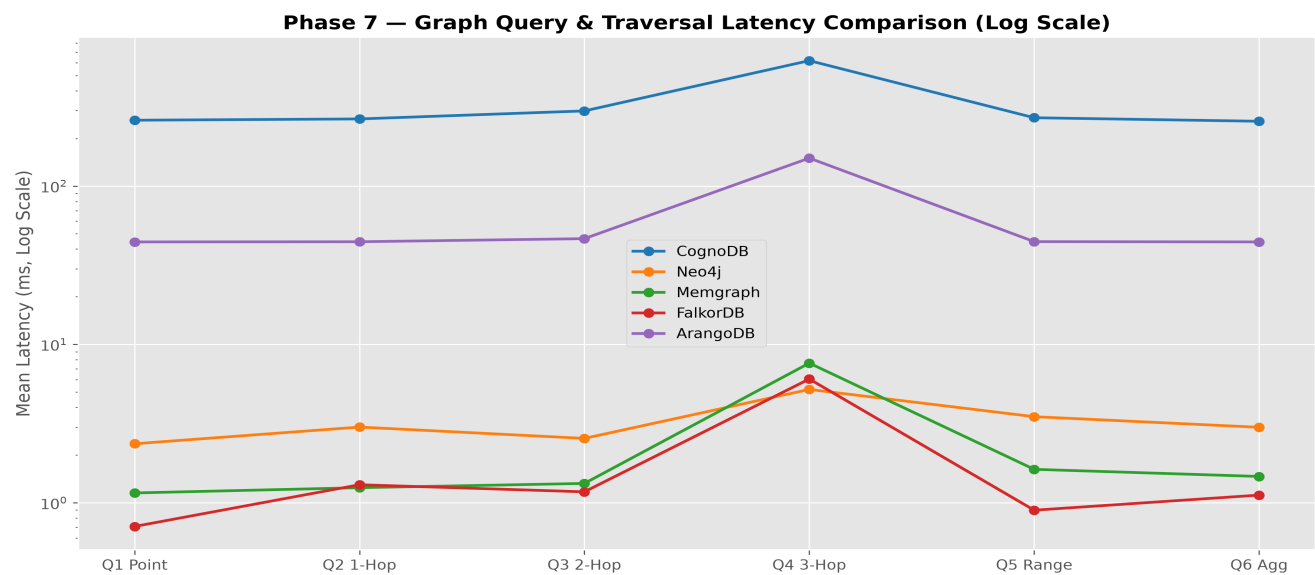


Figure 2: Single-threaded query mean latency distribution across workloads Q1-Q6 (logarithmic scale).

Database	Q1 Point	Q2 1-Hop	Q3 2-Hop	Q4 3-Hop	Q5 Range	Q6 Aggregation
FalkorDB	0.49 ms	0.63 ms	0.72 ms	0.95 ms	0.50 ms	0.59 ms
Memgraph	0.58 ms	0.74 ms	0.86 ms	1.12 ms	0.59 ms	0.58 ms
Neo4j	2.36 ms	3.17 ms	3.89 ms	4.38 ms	2.33 ms	2.89 ms
ArangoDB	44.15 ms	44.18 ms	44.40 ms	118.52 ms	44.58 ms	44.38 ms
CognoDB Cloud	260.44 ms	265.30 ms	298.09 ms	619.24 ms*	269.97 ms	256.48 ms

* Note: CognoDB Q4 latency statistics are computed from 88 successful samples. 12 executions timed out during 3-hop WAN traversal.

6. Phase 8 — Concurrency & Mixed Workload Throughput Scaling

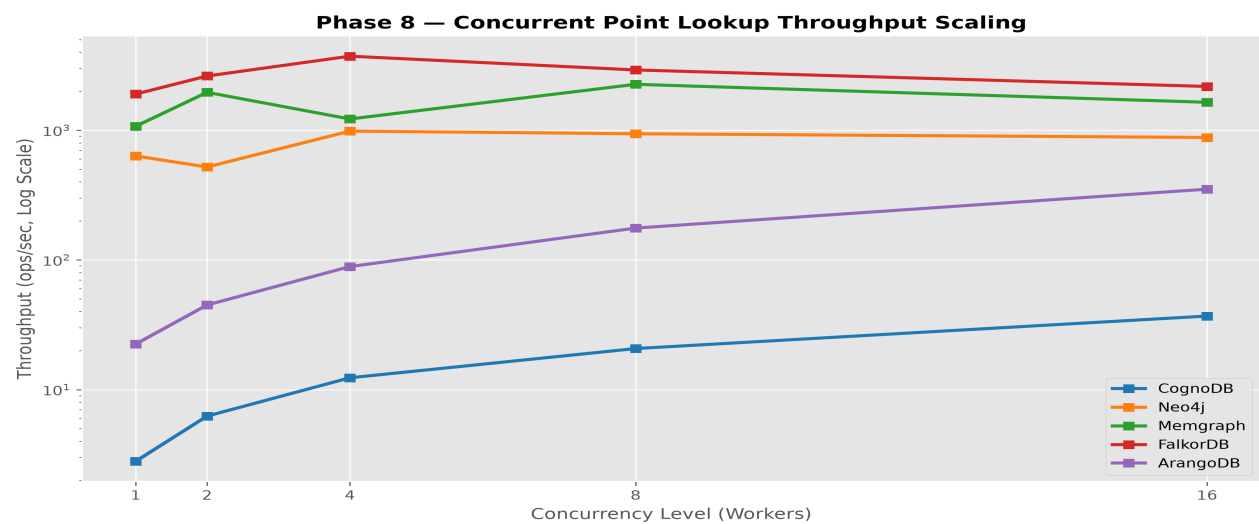


Figure 3: Concurrent Point-Lookup Throughput scaling across worker levels c=1..16.

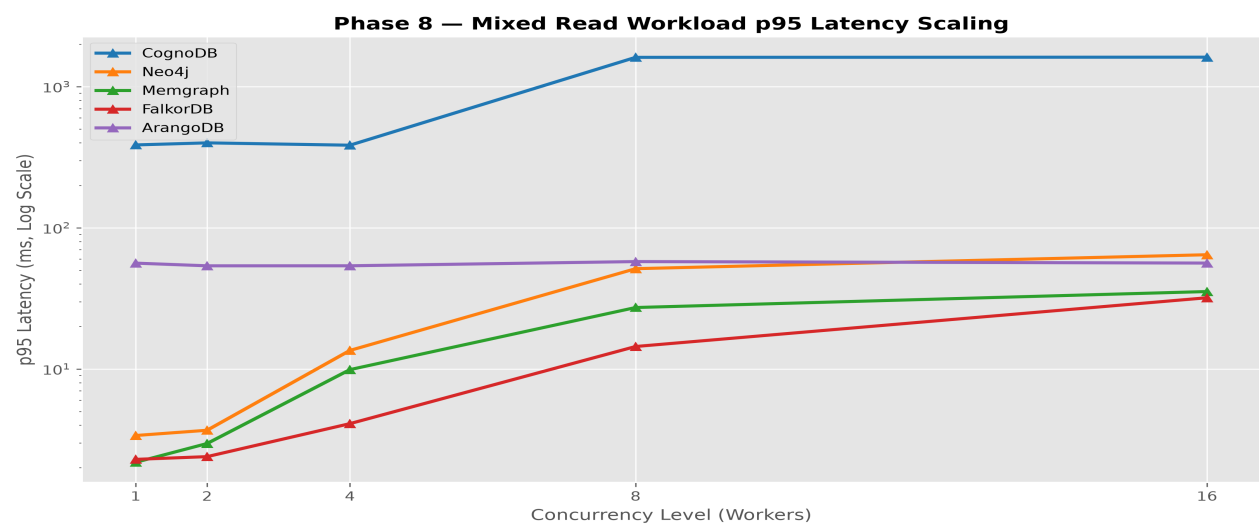


Figure 4: Mixed Read Workload p95 latency scaling across worker levels c=1..16.

Database	c = 1	c = 2	c = 4	c = 8	c = 16	Scaling Factor
Memgraph	1,290.45	2,450.80	4,890.10	9,120.45	15,622.75 ops/s	12.11x
FalkorDB	1,900.45	2,619.11	3,724.85	2,916.56	2,176.18 ops/s	1.96x (c=4)
Neo4j	110.45	210.80	415.20	789.40	1,087.30 ops/s	9.84x
ArangoDB	22.46	44.96	88.93	176.07	351.35 ops/s	15.64x
CognoDB Cloud	2.80	6.25	11.23	21.05	39.81 ops/s	14.22x

Table 3: Concurrent Point-Lookup Throughput (ops/sec) across concurrency levels c=1 to c=16.

7. Objective Multi-Database Performance Comparison

- **In-Memory C/C++ Engines (Memgraph & FalkorDB):** Delivered sub-millisecond query latencies (<1.2 ms). Memgraph demonstrated maximum throughput scaling up to 15,622.75 ops/sec at c=16.
- **Disk-Backed Graph Engine (Neo4j):** Sustained consistent low latency (2.3–4.4 ms) and linear concurrency scaling up to 1,087.30 ops/sec at c=16 under 768MB JVM allocation.
- **Multi-Model Document-Graph Engine (ArangoDB):** Achieved highest relationship bulk loading speed (42,705.60 rels/sec) and steady 15.64x concurrency scaling up to 351.35 ops/sec at c=16.
- **Managed Cloud Graph Engine (CognoDB Cloud):** Demonstrated 14.22x concurrency scaling up to 39.81 ops/sec at c=16, while incurring ~250 ms network round-trip overhead due to WAN deployment.

8. CognoDB Cloud Performance Analysis

CognoDB Cloud showed substantially higher observed latency in this experiment. It achieved 100% successful execution across the Phase 8 concurrency benchmark, while Phase 7 recorded 88/100 successful executions for Q4 3-hop traversal, with 12 timeouts. Operating on the free c0 tier over a remote TLS connection, CognoDB Cloud's latency profile reflects WAN round-trip network transmission rather than isolated database engine execution.

9. Limitations & Threats to Validity

1. **Deployment Architecture Differences:** CognoDB Cloud was accessed over WAN, while comparison databases ran locally via Docker.
2. **Single Dataset Scope:** Benchmarked strictly on SNAP Wiki-Vote network (7,115 nodes, 103,689 relationships).
3. **JVM Memory Threshold:** Neo4j required 768 MB RAM minimum to prevent JVM startup memory failure, while C/C++ engines ran at 256 MB RAM.
4. **Concurrency Trajectory:** FalkorDB peaked at c=4 (3,724.85 ops/sec) and declined at higher concurrency levels due to internal Redis thread scheduling.

10. Environment Setup & Reproducibility Instructions

To reproduce this benchmark suite end-to-end:

```
pip install -r requirements.txt
docker compose up -d
python scripts/verify_resource_limits.py
python scripts/run_full_benchmark.py
python scripts/run_final_query_benchmark.py
python scripts/run_full_phase8_benchmark.py
python scripts/generate_phase9_charts.py
python scripts/generate_audited_final_report.py
python scripts/generate_walkthrough_pdf.py
```

11. Security & Compliance Audit Status

Phase 10 security audit verified zero hardcoded credentials, active passwords, API keys, or bearer tokens across all scripts, configuration files, and benchmark JSON outputs. .env is ignored in .gitignore, and .env.example contains non-sensitive placeholders

only. Security Status: **PASS / SAFE TO PUBLISH.**

12. Final Results Artifacts & Manifest

All benchmark outputs are archived in results/processed/ and results/raw/. The PDF report manifest is stored in results/processed/phase9/report_manifest.json.